

CSE3016 - AWS SOLUTION ARCHITECT

UNIT IV

Database: RDS Multi-AZ Deployments, RDS Read Replicas, Amazon Aurora, Amazon DynamoDB, DynamoDB Streams

Presented By

Dr Jasmine Selvakumari Jeya I

Senior Associate Professor Grade 2/SCOPE

VIT Bhopal University

Module IV

Database – RDS Multi-AZ Deployments, RDS Read Replicas, Amazon Aurora, Amazon DynamoDB, DynamoDB Streams

Management Tools – AWS Management Tools, AWS CloudWatch, CloudWatch Metrics, CloudWatch Alarms, AWS CloudTrail, AWS Config, AWS Systems Manager.

Management & Governance – AWS ELB and Auto Scaling, CloudFormation, CloudTrail, CloudWatch.

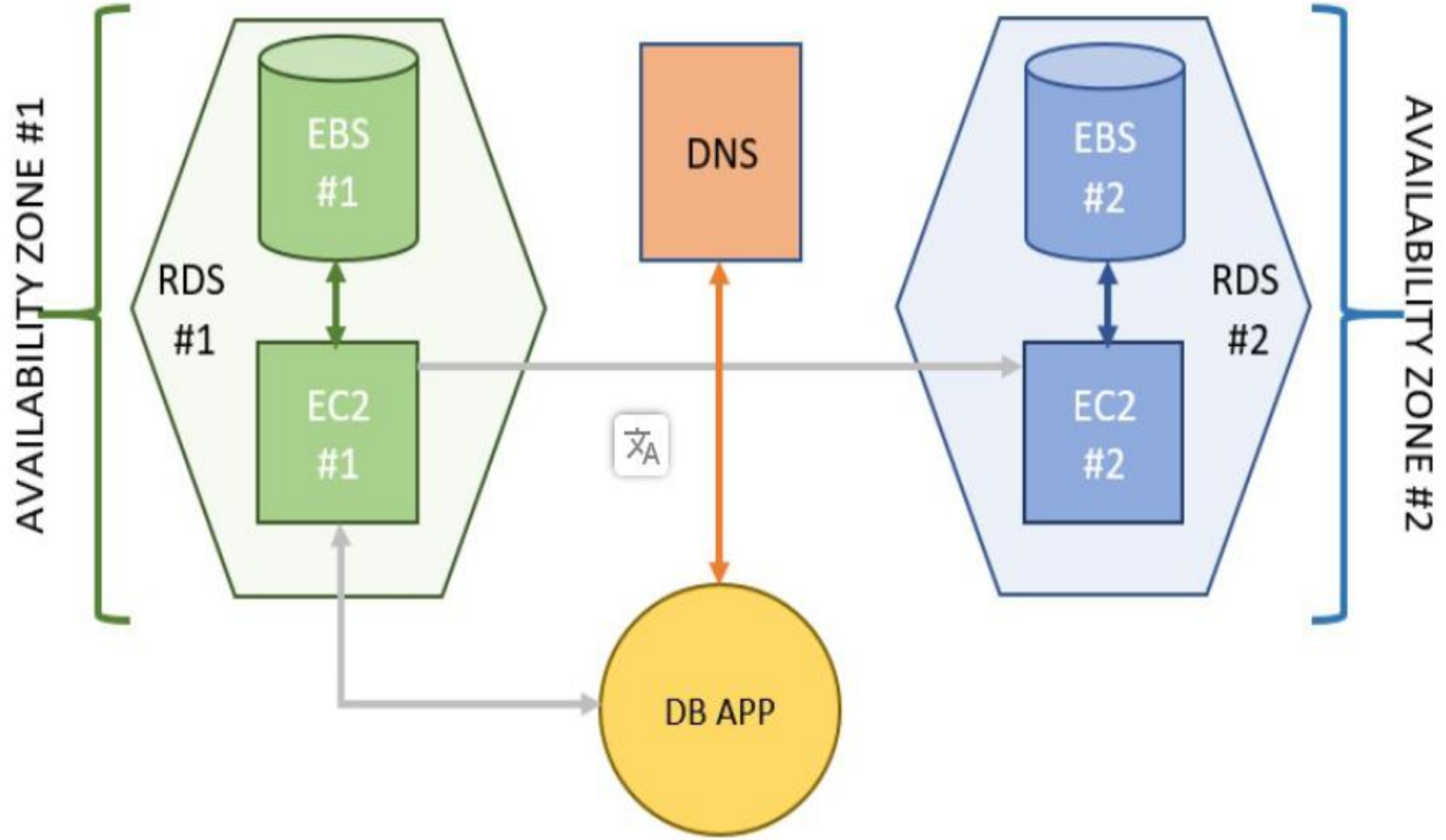
Database – RDS Multi-AZ
Deployments, RDS Read Replicas,
Amazon Aurora,
Amazon DynamoDB, DynamoDB
Streams

Amazon RDS

- Amazon Relational Database Service (RDS) is a fully managed cloud service designed to simplify setting up, operating, and scaling relational databases.
- Provides cost-efficient and resizable database capacity for industry-standard relational database systems.
- Automates routine database administration tasks such as provisioning, OS and database patching, backups, recovery, failure detection, and automatic repair.
- Eliminates the need to manually manage physical servers, operating systems, and database software.
- Designed to improve reliability, performance, and availability of database workloads in the cloud.

Supported Database Engines

- Amazon Aurora (AWS-built, high-performance engine)
- PostgreSQL
- MySQL
- MariaDB
- Oracle Database
- Microsoft SQL Server



Core Architecture: Multi-AZ vs. Read Replicas

1. Multi-AZ Deployment (High Availability)

- **Purpose:** Ensures High Availability and Disaster Recovery (DR).
- **Operation:** RDS provisions a **standby replica** in a different Availability Zone (AZ).
- **Replication Type:** Uses **synchronous replication** to keep the standby up to date.
- **Failover:** In case the primary DB instance fails, RDS automatically **fails over** to the standby instance.
- **Endpoint Handling:** The **database endpoint remains unchanged**, so applications continue working without configuration updates.
- **Usage:** The standby instance **cannot be used** for read or write operations; it stays idle until a failure occurs.

2. Read Replicas (Scalability)

- **Purpose:** Enhance performance by offloading read-heavy workloads from the primary database.
- **Operation:** Creates a **read-only replica** of the primary database.
- **Replication Type:** Uses **asynchronous replication** from the primary DB to the Read Replica.
- **Usage:** Applications can connect to Read Replicas for read operations such as analytics, reporting, and dashboards, reducing load on the primary instance.
- **Promotion:** A Read Replica can be **promoted to a standalone database** when needed, making it function independently from the primary.

The Shared Responsibility Model for Amazon RDS

AWS Responsibility – “Security OF the Cloud”

- Ensuring physical security of AWS data centers.
- Managing and maintaining the underlying host infrastructure (e.g., EC2, EBS).
- Handling OS-level patching and database engine software patching.
- Providing automated backups, snapshots, and underlying storage replication.

Customer Responsibility – “Security IN the Cloud”

- **Data Management:** Controlling and managing the data stored inside the database.
- **Encryption:** Enabling and configuring encryption at rest (KMS) and encryption in transit (SSL).
- **Network Security:** Setting up and maintaining Security Groups (firewalls) to restrict inbound/outbound access (e.g., allowing only App Server access on port 3306).
- **IAM Management:** Defining which AWS users/roles can access and manage RDS resources through the AWS console or APIs.
- **Database User Management:** Creating, managing, and securing database users, roles, and permissions within the DB engine.

Advanced Features of Amazon RDS

1. RDS Proxy

- Designed for modern applications (especially serverless apps like AWS Lambda) that create large numbers of database connections.
- Prevents database overload caused by excessive or rapid connection spikes.
- Works as a **connection pooling layer** between your application and the database.
- Reuses and manages connections efficiently, improving performance and stability.
- Enhances database resilience during traffic surges and failovers.

2. Blue/Green Deployments

- Addresses risks associated with database schema updates, engine upgrades, or parameter modifications.
- Creates a **Green environment** that is an identical clone of the existing **Blue production environment**.
- You can safely test upgrades, changes, and performance on the Green environment.
- Switchover from Blue to Green typically occurs in **under a minute**, with **zero data loss**.
- Ensures safe, fast, and predictable database updates.

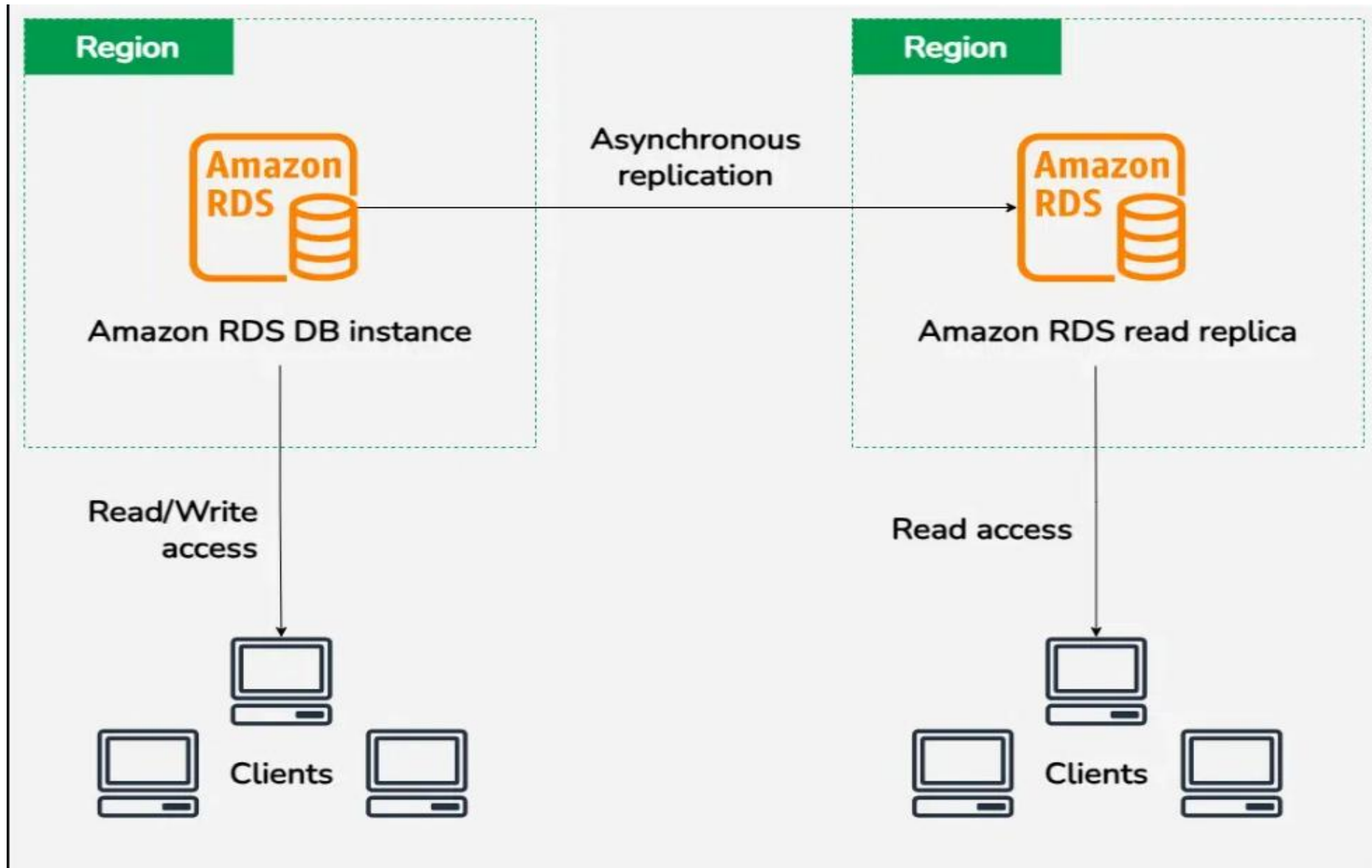
3. Storage Auto-Scaling

- Eliminates the need to over-provision storage in advance.
- Automatically monitors free storage space in your RDS instance.
- When storage is nearly full, RDS automatically expands the allocated storage.
- Scaling happens **on the fly** with **zero downtime** or service interruption.
- Ensures smooth performance even under unexpected data growth.

Understanding AWS Regions, Availability Zones, and Multi-AZ Deployments in Amazon RDS

AWS Regions

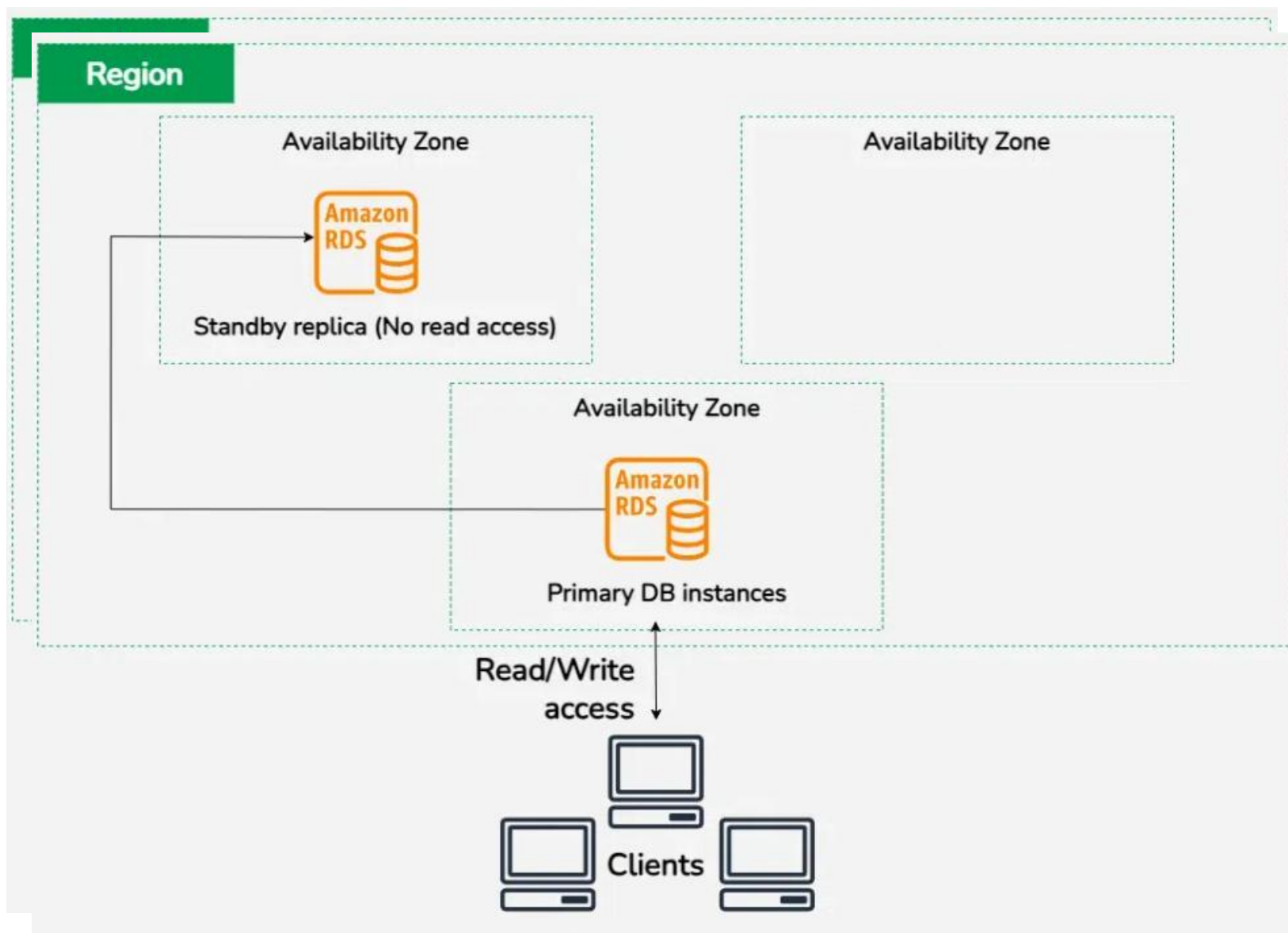
- Geographically separated locations across the world.
- Each region operates independently to provide geographic redundancy.
- When you create an RDS instance, the **chosen region determines where the database is physically hosted.**
- **Availability Zones (AZs)**
 - Multiple, isolated data centers within a single AWS Region.
 - Each AZ has **independent power, cooling, and networking**, improving fault tolerance.
 - Provide higher availability and resilience compared to using a single data center.



This configuration delivers a resilient architecture that enables global application scaling while maintaining strong data redundancy and enhanced fault tolerance.

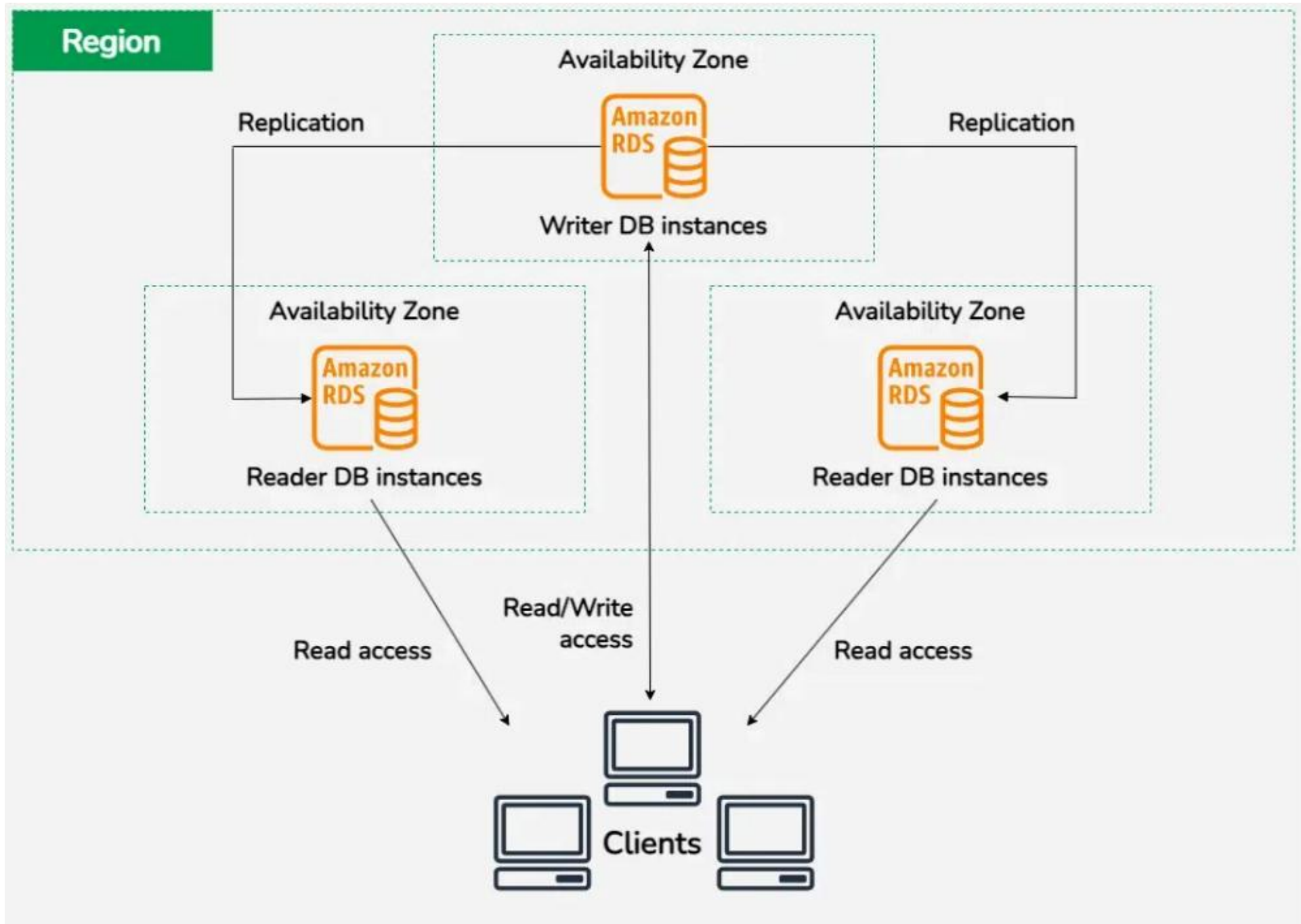
Multi-AZ Deployments

- In Multi-AZ deployments, Amazon RDS automatically replicates the primary database to a standby instance in a different Availability Zone within the same region.
- The **primary instance** handles all read/write operations.
- The **standby instance** continuously stays in sync through synchronous replication for disaster recovery.
- If the primary instance or its AZ fails, RDS automatically performs a **failover** to the standby instance.
- This ensures **minimal downtime** and maintains high availability.
- The setup provides enhanced reliability and fault tolerance across multiple AZs.



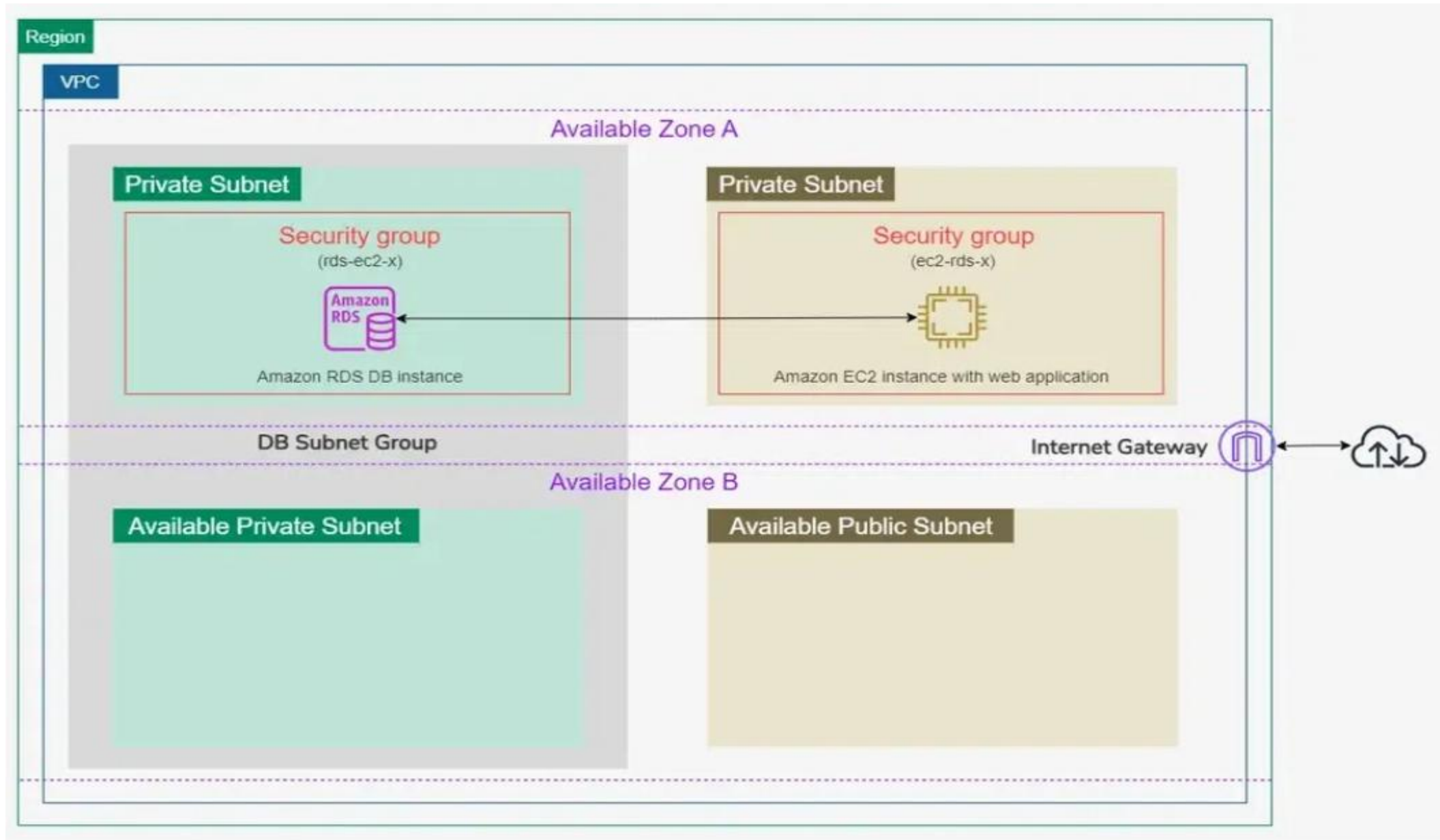
- A Multi-AZ DB cluster includes **one writer DB instance** and **two reader DB instances**.
- All three instances are distributed across **three different Availability Zones** within the same AWS Region.
- The **writer instance** processes both read and write operations.
- The **reader instances** are dedicated exclusively to handling read traffic.
- This architecture enhances availability, improves read scalability, and provides fault tolerance across multiple AZs.

Region



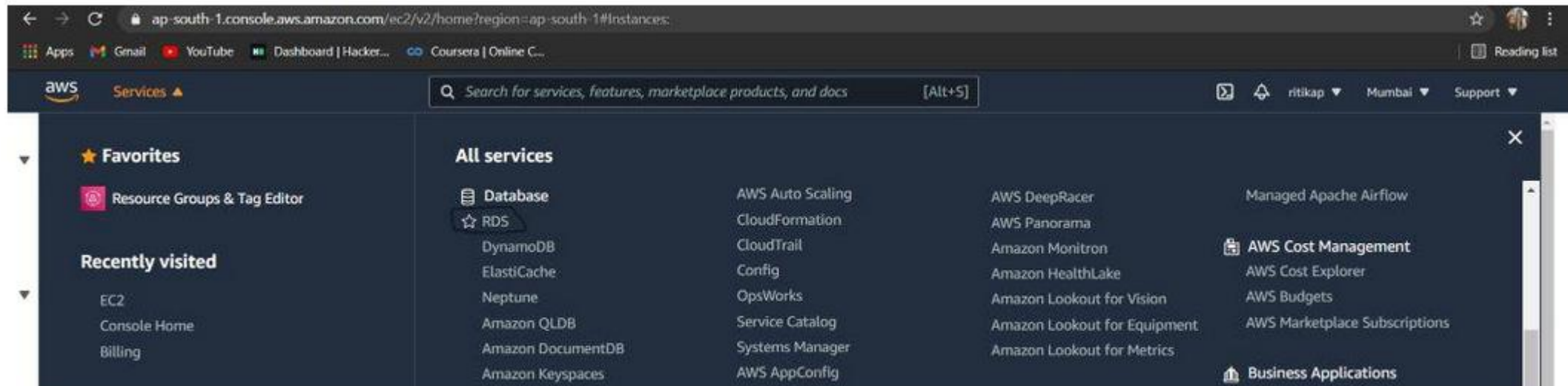
Access Control with Security Groups for RDS DB Instances

- Security groups help control network access between EC2 instances and RDS DB instances in a VPC.
- They act as virtual firewalls that manage inbound and outbound traffic.
- The RDS DB instance's security group decides which EC2 instances can connect to the database.
- EC2 instances also have their own security groups to define allowed connections.
- The architecture clearly shows how EC2 and RDS security groups are configured within the VPC.
- Only allowed traffic (such as database port access) is permitted between EC2 and RDS.
- This setup ensures secure and controlled communication inside the VPC.



Ways to Interact with Amazon RDS

- You can interact with Amazon RDS in multiple ways
- **AWS Management Console**
- The AWS Management Console provides an easy-to-use web-based interface to manage and monitor your RDS instances including creating databases, managing backups and scaling instances



Command line interface

- The AWS CLI enables users to automate tasks by executing commands in scripts. For more information Please Refer to this [Configure CLI for RDS](#)

Amazon RDS APIs

- Amazon RDS API allows developers to programmatically manage and interact with their RDS instances offering seamless integration into applications and services.

Drawbacks of Amazon RDS

- Limited customization because RDS is a fully managed service.
- Can become expensive for large deployments or high-capacity instances.
- Scaling is not fully automatic and needs manual adjustments.
- Backups and restores may take longer during heavy load.
- Migrating away from RDS can be difficult, causing vendor lock-in.

Amazon RDS Database Instances

- RDS database instances are managed virtual servers for running relational databases in AWS.
- They come pre-configured with the required hardware and software for your chosen database engine.
- Supported engines include MySQL, PostgreSQL, Oracle, SQL Server, and MariaDB.
- RDS automatically handles backups, patching, and routine database management tasks.
- You can select different instance sizes based on performance requirements.
- Instances can be scaled up or down depending on workload needs.
- Multi-AZ deployment options provide high availability and fault tolerance.

AWS Cloud

Virtual private cloud (VPC)

Availability Zone 1



Oracle Instance



Oracle Instance

Availability Zone 2



PostgreSQL instance



MySQL instance

Amazon Aurora and Amazon Aurora Serverless vs Amazon RDS

Feature	Amazon Aurora	Amazon Aurora Serverless	Amazon RDS
Performance	Optimized for high performance and scalability	Automatically scales based on workload demand	Good performance but may have limitations for extremely high transaction loads
Scalability	Automatic storage scaling up to 128 TB	Automatically scales resources up/down based on demand	Manual scaling required for instance and storage resources
High Availability	Multi-AZ replication for fault tolerance	Multi-AZ replication available, scales with demand	Multi-AZ option available but requires manual setup
Cost	Pay for provisioned instances and storage	Pay only for the capacity used — cost-effective for variable workloads	Cost-effective for small workloads but requires manual scaling adjustments

Amazon Aurora

Amazon Aurora

- Amazon Aurora is a fully managed relational database engine by AWS.
- It is compatible with both MySQL and PostgreSQL.
- Aurora offers performance similar to high-end commercial databases.
- It remains cost-effective like open-source databases.
- Provides up to **5x faster performance than MySQL** and **3x faster than PostgreSQL**.
- It is part of the Amazon RDS family.
- Uses a cloud-native architecture that **separates compute and storage** for better scalability and reliability.

**Writer Instance
(Primary)**

Failover Target

Reader Instance (Read Replica)

Reader Instance (Read Replica)

Reader Instance (Read Replica)

Shared Storage Volume



AZ1



AZ2



AZ3



Aurora Storage Architecture

- Aurora uses a **distributed, log-structured storage system**, not a local disk like traditional databases.
- **6 copies** of data are stored across **3 Availability Zones (AZs)**.
- Can tolerate **2 copy failures** without affecting writes and **3 copy failures** without affecting reads.
- The storage system is **self-healing**, automatically repairing corrupted data blocks.
- Storage is **shared** by all compute instances, allowing up to **15 read replicas** without replication lag

Aurora Architecture Components

- Aurora clusters have **two parts: a Writer (Primary) and Reader Instances.**
- **Writer Instance:**
 - Handles all write and read operations.
 - Only **one primary instance** per cluster.
- **Reader Instances:**
 - Handle read-only queries.
 - Up to **15 replicas** supported.
 - Automatically promoted to primary during failover (usually < 30 seconds).

Aurora Deployment Modes

1. Aurora Provisioned

- You choose a specific instance type manually.
- Best for **predictable workloads**.
- Scaling requires manual changes or adding replicas.

2. Aurora Serverless v2

- You define a **capacity range** (ACUs).
- Automatically scales compute and memory within seconds.
- Best for **variable or unpredictable workloads**.
- You pay only for the **ACUs used** per second.

Advanced Features

1. Aurora Global Database

- Supports **multi-region** deployments.
- Replication latency is usually **under 1 second**.
- Secondary regions can be promoted in < 1 minute for DR.
- Users read from the region nearest to them for low latency.

2. Backtracking

- Allows you to **rewind the database** to a past moment instantly.
- Useful for mistakes like accidental deletes or bad deployments.

3. Fast Cloning

- Creates new clusters **within minutes**, regardless of data size.
- Uses **copy-on-write**, so storage is shared until changes occur.
- Useful for testing or analytics environments.

How Amazon Aurora Works

- Aurora clusters have a **primary instance**, **replica instances**, and **shared cluster storage**.
- Data is stored across multiple AZs for durability.
- Primary instance handles **reads + writes**.
- Up to **15 replicas** handle read traffic and improve availability.
- Failover automatically switches to a replica when primary fails.
- Supports **multi-master**, where multiple instances can read/write.
- You can store backups automatically in **Amazon S3**.
- Aurora Serverless adjusts capacity automatically for unpredictable workloads.

Amazon RDS Shared Responsibility Model

AWS Responsibilities

- Manages **data centers, hardware, networking, and physical security.**
- Handles **database software installation, patching, and maintenance.**
- Automates **backups, storage scaling, replication, and Multi-AZ availability.**
- Ensures **infrastructure-level security and compliance.**

Customer Responsibilities

- Must configure and tune database settings for performance.
- Responsible for **data security, encryption, and access control** via IAM.
- Must monitor **queries, performance, and workloads** using tools like CloudWatch and Performance Insights.

Aurora Vs RDS (MySQL/PostgreSQL)

Why choose Aurora over standard RDS?

Feature	Amazon Aurora	Standard RDS (MySQL/PostgreSQL)
Performance	Up to 5x faster (MySQL) / 3x faster (Postgres)	Standard engine performance
Storage Scaling	Auto-scales up to 128 TB	Manual scaling (up to 64 TB)
Replication	< 10 ms lag (Shared Storage)	Seconds/Minutes lag (Binlog replication)
Failover Time	< 30 seconds	60–120 seconds
Replicas	Up to 15	Up to 5
Cost	~20% higher per instance hour	Lower instance cost

Amazon Aurora Pricing

- Aurora pricing has three main components:

1. Instance/Compute:

- **Provisioned:** Hourly rate based on instance size.
- **Serverless:** Per ACU-hour (billed per second).

2. Storage: Pay per GB-month for the data you store (auto-expanding).

3. I/O Rate:

- **Aurora Standard:** Pay per million I/O requests. (Best for low-I/O apps).
- **Aurora I/O-Optimized:** Higher storage/instance cost, but **zero** charges for I/O. (Best for I/O-intensive apps like payment processors).

Amazon DynamoDB

DynamoDB

- DynamoDB lets users create databases that can store and retrieve **any amount of data**.
- It can handle **any level of traffic** by automatically distributing data and requests across servers.
- It is a **fully managed NoSQL database service** with fast, predictable performance.
- DynamoDB provides **automatic and seamless scalability**.
- Users do not need to manage hardware, patch software, or scale clusters manually.
- It reduces administrative work by handling all operational tasks automatically.
- It protects sensitive data by offering **encryption at rest** without additional setup.

1. Creating a DynamoDB Table

1. Sign in to AWS Management Console

- Navigate to **DynamoDB** under the AWS Services.

2. Click “Create Table”

- Start the table creation wizard.

3. Define Table Name

- Enter a **unique table name** that identifies your data.

4. Define Primary Key

- **Partition Key (hash key):** A unique attribute that determines how data is distributed across partitions.
- **Sort Key (optional, range key):** Allows multiple items with the same partition key to be stored in order.

5. Configure Table Settings

- **Provisioned vs. On-Demand Capacity:**
 - *Provisioned*: Set read/write capacity units.
 - *On-Demand*: Automatically scales based on traffic.
- **Encryption**: Enable server-side encryption (SSE) for data security.
- **Secondary Indexes (optional)**: For querying data in different ways (Global Secondary Index or Local Secondary Index).

6. Advanced Settings (Optional)

- **TTL (Time To Live):** Automatically delete expired items.
- **Stream Settings:** Enable DynamoDB Streams for real-time data changes.

7. Create Table

- Review all configurations and click **Create**.
- The table is now available for storing and querying data.

Create table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.



1 to 255 characters and case sensitive.

Sort key - optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.



1 to 255 characters and case sensitive.

Table settings

☒ Default settings

The fastest way to create your table. You can modify these settings now or after your table has been created.

☐ Customize settings

Use these advanced features to make DynamoDB work better for your needs.

2. Key Attributes to Define

- **Partition Key:**

- Determines **how data is distributed** across partitions.
- Must be chosen carefully for even data distribution.

- **Sort Key (optional):**

- Enables **composite primary keys** (partition + sort key).
- Useful for querying ranges or ordered datasets.

- **Indexes:**

- Secondary indexes allow **efficient queries** without scanning the entire table.

- **Capacity Mode:**

- Impacts cost, performance, and scalability.

3. Importance of Choosing the Right Keys

- **Partition Key:**

- Poor choice can lead to **hot partitions**, causing performance bottlenecks.
- Good choice ensures **even data distribution** and high throughput.

- **Sort Key:**

- Enables **efficient querying** of related items.
- Important for datasets with **one-to-many relationships**, e.g., user orders.

4. Use Case for DynamoDB

Scenario: A real-time gaming leaderboard system

- **Requirements:**

- Thousands of users updating scores simultaneously.
- Fast retrieval of top scores by player or region.

- **Why DynamoDB?**

- **Scalability:** Handles millions of read/write requests per second.
- **Performance:** Single-digit millisecond latency ensures smooth gameplay.
- **Flexible querying:** Using partition key as player ID and sort key as timestamp or score.

AWS DynamoDB - Working with Streams

DynamoDB Streams

- DynamoDB Streams keeps track of **all changes** made to items in a DynamoDB table.
- It provides an **ordered flow of change records** such as inserts, updates, and deletes.
- Useful for use cases like running **validation or processing** whenever user data changes.
- Streams allow applications to read **near-real-time logs** of table changes.

Features of DynamoDB Streams

- Stores stream data for **up to 24 hours**, so applications must process it within that time.
- If the DynamoDB table is encrypted, the stream is **also encrypted**.

Create tableDelete table

Filter by table name

Choose a table ...

Actions

Name

geeksforgeeks

geeksforgeeks Close

Overview

Items

Metrics

Alarms

Capacity

Indexes

Global Tables

Backups

Contributor Insights

Triggers

Access control

Tags

Recent alerts

No CloudWatch alarms have been triggered for this table.

Kinesis data stream details

Use Amazon Kinesis Data Streams for DynamoDB to capture item-level changes in your table as a Kinesis data stream. [Learn more](#)

Stream enabled No

Stream name -

Manage streaming to Kinesis

DynamoDB stream details

Stream enabled No

View type -

Latest stream ARN arn:aws:dynamodb:us-east-1:123456789012:stream/2021-07-26T08:59:55.645Z/geeksforgeeks/stream/2021-07-26T08:59:55.645Z

Manage DynamoDB stream

Table details

Table name	geeksforgeeks
Primary partition key	UserID (String)
Primary sort key	-
Point-in-time recovery	DISABLED Enable
Encryption Type	DEFAULT Manage Encryption
KMS Master Key ARN	Not Applicable
Encryption Status	-
CloudWatch Contributor Insights	DISABLED Manage Contributor Insights NEW
Time to live attribute	DISABLED Manage TTL
Table status	Active
Creation date	July 26, 2021 at 2:29:41 PM UTC+5:30
Read/write capacity mode	Provisioned
Last change to on-demand mode	-
Provisioned read capacity units	5 (Auto Scaling Disabled)
Provisioned write capacity units	5 (Auto Scaling Disabled)
Last decrease time	-
Last increase time	-
Storage size (in GB)	0 GB

Enable streams on a DynamoDB table

- *Follow the below steps to enable streams on the DynamoDB table:*
- **Step 1:** Open the AWS console. Navigate to DynamoDB Dashboard -> Choose tables -> Select the existing table you want to enable DynamoDB streams for. Click on Manage DynamoDB Streams Button as shown below.
- **Step 2:** Chose the appropriate option for your use case and click on Enable.
- **Keys only** - Only the key attributes of the modified item are received in the stream.
- **New image** - The entire updated item is received, as it appears **after** the update happened.
- **Old image** - The entire updated item is received, as it appears **before** the update happened.
- **New and old images** - Both records of the updated item are received, as it appears **before and after** the update happened.

- This will enable DynamoDB streams which will receive logs of all the changes made to data in DynamoDB. Please note that if you perform PUT or UPDATE operations in the table and no item is modified the operation logs/data are not recorded in DynamoDB streams.

Manage Stream ×

View type

- ☐ Keys only - only the key attributes of the modified item
- ☐ New image - the entire item, as it appears after it was modified
- ☐ Old image - the entire item, as it appeared before it was modified
- ☒ New and old images - both the new and the old images of the item

Cancel Enable

Using Data in DynamoDB streams

- **DynamoDB Streams — How Data Is Processed**

- DynamoDB Streams provides **near real-time data** about all changes made to a DynamoDB table.
- To read and process stream data, you can build a **custom application** that connects to the DynamoDB Streams endpoint using API requests.
- You can also use **AWS Lambda** by creating a DynamoDB trigger.
- Whenever new data is added to the stream, **Lambda is automatically invoked** to process the change.
- If you need, I can also make this into a diagram or flowchart.

Adding a lambda trigger

Lambda > Add trigger

Add trigger

Trigger configuration

 **DynamoDB**
aws database nosql

DynamoDB table
Choose or enter the ARN of a DynamoDB table.

Q am:aws:dynamodb:us-east-1 [redacted] table/geeksforgeeks X ↺

Batch size
The largest number of records that will be read from your table's update stream at once.

100

Batch window - optional
The maximum amount of time to gather records before invoking the function, in seconds.

Starting position
The position in the stream to start reading from. For more information, see [ShardIteratorType](#) in the Amazon DynamoDB Streams API Reference.

Latest

► **Additional settings**

In order to read from the DynamoDB trigger, your execution role must have proper permissions.

☒ **Enable trigger**
Enable the trigger now, or create it in a disabled state for testing (recommended).

Cancel Add

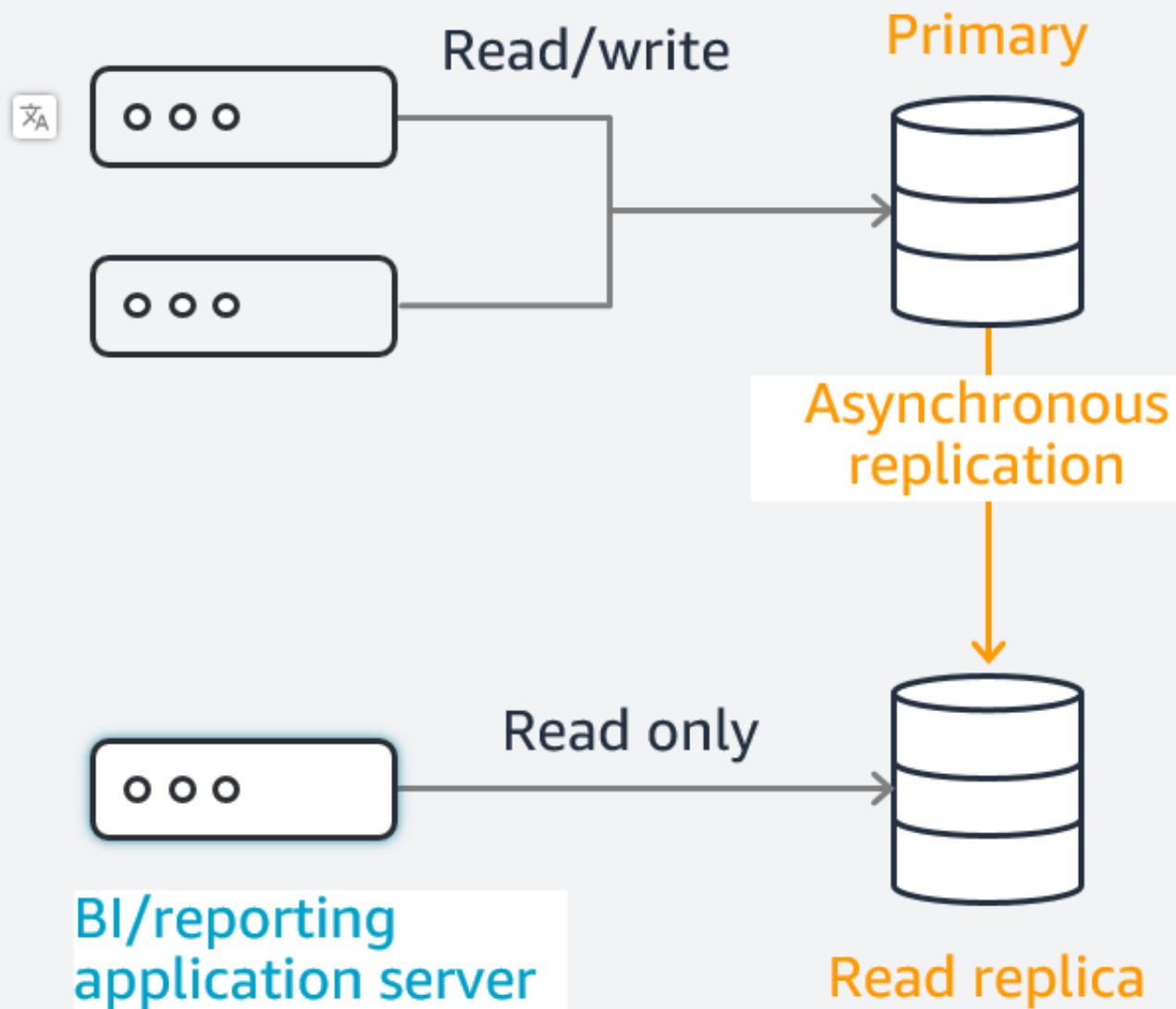
Amazon RDS - Working with Read Replicas

Read Replicas

- Read replicas are **non-editable copies** of the main (primary) database.
- They are created using **asynchronous replication** from the primary RDS instance.
- Any updates in the primary database are **automatically reflected** in the replicas.
- Replicas can be placed in the **same region or different regions**.
- Having replicas in multiple regions helps with **faster recovery** and improved read performance.
- Most database engines support the basic concept of read replication.
- Diagrams (like the one provided) help visualize how replicas work with the primary database.

Application servers

Database server



Advantages of Read Replicas

- **Performance Improvement:**

- Read replicas handle all read queries, reducing load on the primary database.
- Helps improve overall RDS performance.
- Region-specific replicas can offer faster read performance for users in different locations.

- **Higher Availability:**

- Helps during hardware failures or disasters.
- A replica can be promoted to a standalone database to replace a failed primary instance.
- Keeps the system running instead of a complete downtime.

- **Security:**

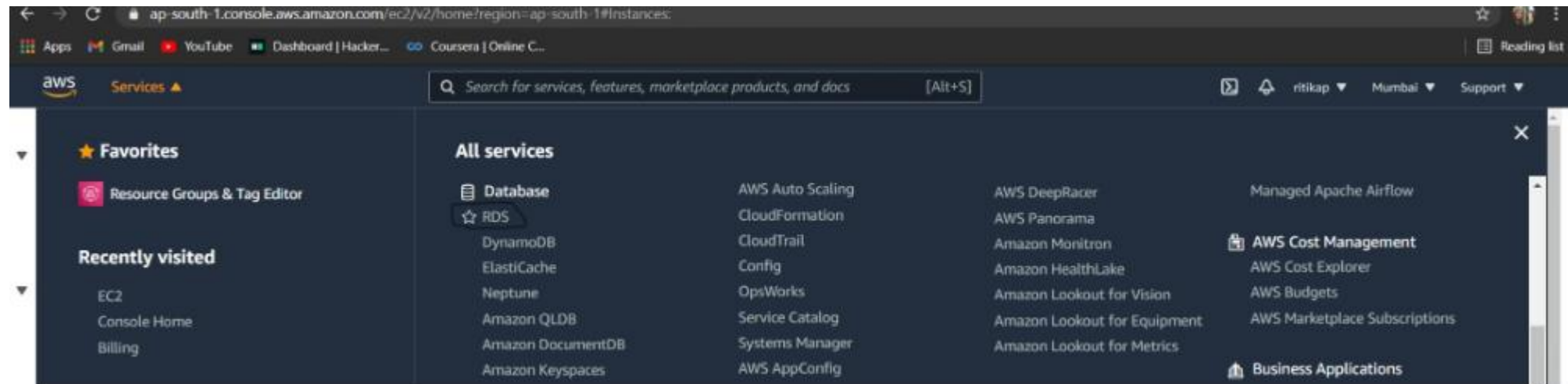
- RDS automatically encrypts data using public keys when creating replicas.
- Ensures secure data handling and protection.
- Additional security layers can be added based on requirements.

Notable Features of Read Replicas

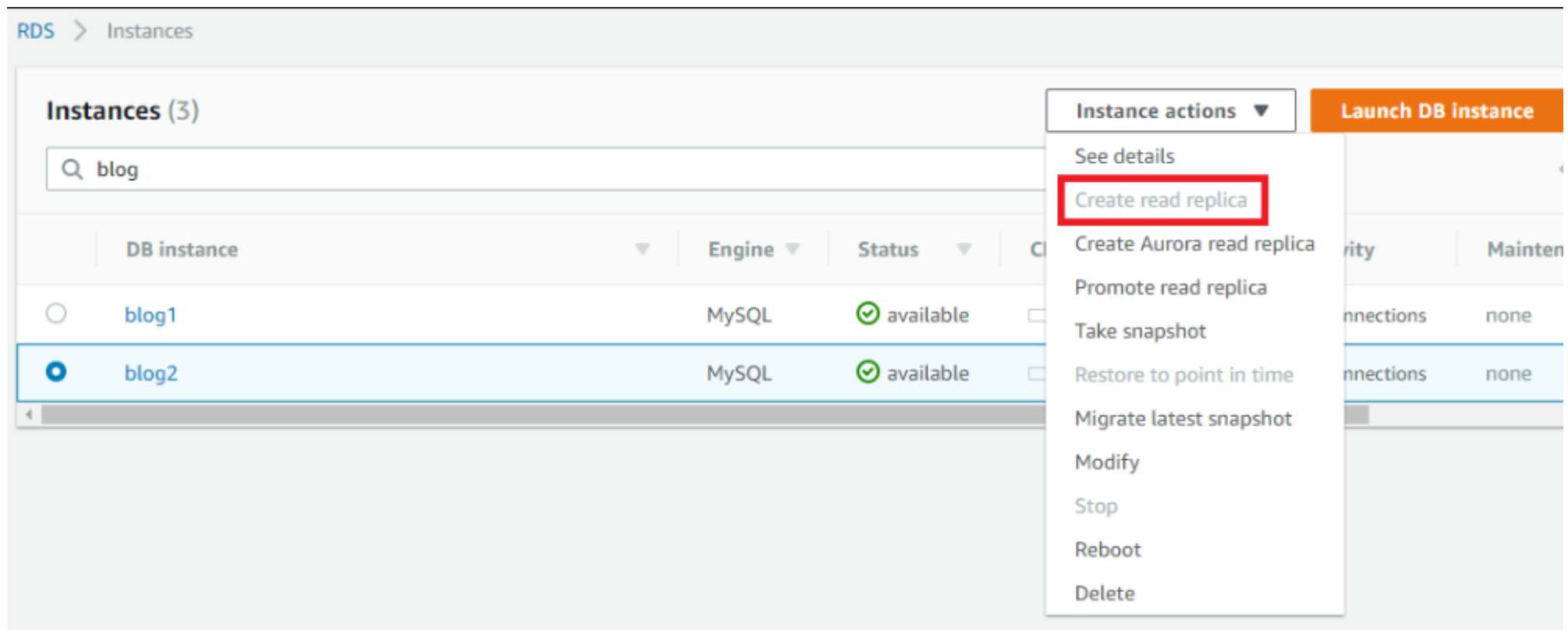
- You can create **up to 5 read replicas** for a database.
- Large organizations can create **replicas of read replicas**, though this may add latency.
- Replicas can be **converted into standalone databases** to support heavy workloads.
- During a primary database failure, **one replica can be promoted as the new primary**, reducing data loss.

- *Let us know about the steps involved in creating a read replica.*
- **Note:** As we all know, the concept of replicas is useful for huge organizations as they have a large database to work with. Thus, [AWS](#) has not allotted this facility for free tier accounts as we individuals do not need this service.

Step 1: Login into your AWS account. From services select RDS. Image attached ahead to refer to.



Step 2: Select the database you want to make read replicas of. And from "**Actions**" choose "**Read Replicas**". Here is the image attached to refer to.



Step 3: Once you are directed ahead. Align all the configurations as per your requirements. Refer to the image attached ahead.

RDS > Databases > Add reader

Create read replica DB instance

You are creating a replica DB instance from a source DB instance. This new DB instance will have the source DB instance's DB security groups and DB parameter groups.

Network & Security

Destination region
The region in which the replica will be launched

Asia Pacific (Mumbai) ▼

Destination DB subnet group

default ▼

Availability zone
The EC2 Availability Zone that the database instance will be created in.

ap-south-1b ▼

Publicly accessible

☐ Yes
EC2 instances and devices outside of the VPC hosting the DB instance will connect to the DB instances. You must also select one or more VPC security groups that specify which EC2 instances and devices can connect to the DB instance.

☒ No
DB instance will not have a public IP address assigned. No EC2 instance or devices outside of the VPC will be able to connect.

Step 4: When you find everything appropriately configured as per the needs. Click on "Create Read Replica". Refer to the image attached ahead for understanding better.

Performance Insights

☐ Enable Performance Insights

☒ Disable Performance Insights

Maintenance

 Auto minor version upgrade

Specifies if the DB instance should receive automatic engine version upgrades when they are available.

☐ Yes

☒ No

Cancel

Create read replica